

Exam Logistics:

- Answer any 10 of 11 questions. If all 11 are attempted, only the first 10 will be corrected and considered for the total score.
- Each question is worth 5 marks, and requires roughly 15-20 minutes' worth of effort.
- The final exam accounts for 50% of the final score.

Instructions for the Exam:

- Use appropriate indentation when writing pseudocode. This is strongly encouraged.
- What is expected when providing an algorithm:
 1. Algorithm must be written as pseudocode, and not descriptive format.
 2. Add comments wherever necessary to convey your ideas clearly.
 3. Do not provide code in any specific programming language.
 4. The following must be provided whenever asked to analyze an algorithm:
 - (a) Proof of correctness.
 - (b) Justification of running time.
 - (c) As required, space complexity.

Questions (5 marks each):

1. See the given network in Figure 1, with edge capacities.

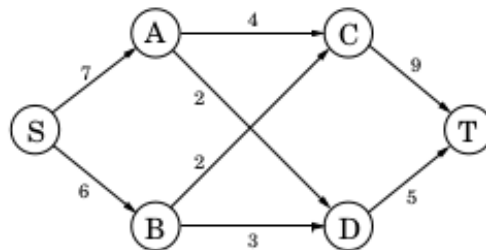


Figure 1: A given network to solve for max-flow problem

- (a) Find the maximum flow f and a minimum cut.
- (b) Draw the residual graph G_f (along with its edge capacities). In this residual network, mark the vertices reachable from S and the vertices from which T is reachable.
- (c) An edge of a network is called a *bottleneck edge* if increasing its capacity results in an increase in the maximum flow. List all bottleneck edges in the above network.
- (d) Give a very simple example (containing at most four nodes) of a network which has no bottleneck edges.

2. *Subset sum* – Give an $O(nt)$ algorithm for the following task.

Input: A list of n positive integers $a_1, a_2, \dots, a_n$; a positive integer t .

Question: Does some subset of the a_i s add up to t ? (a_i can be used at most once in the sum.)

(*Hint:* Look at subproblems of the form “does a subset of $\{a_1, a_2, \dots, a_i\}$ add up to s ?”)

3. The goal is to determine the amortized cost per operation for a sequence of n operations on a data structure in which the i^{th} operation costs i if i is an exact power of 2, and 1 otherwise. Show the amortized cost computation using:
- (a) Aggregate analysis,
 - (b) Accounting method.
4. Prove the following properties of the Huffman encoding scheme.
- (a) If some character occurs with the frequency more than $\frac{2}{5}$, then it is guaranteed to be a codeword of length 1.
 - (b) If all characters occur with frequency less than $\frac{1}{3}$, then there is guaranteed to be no codeword of length 1.
5. **STINGY SAT** is the following problem: given a set of clauses (each a disjunction of literals) and an integer k , find a satisfying assignment in which at most k variables are true, if such an assignment exists. Prove that **STINGY SAT** is **NP**-complete.
6. In **HIRE-ASSISTANT**, assuming that the candidates are presented in a random order, what is the probability that one hires exactly one time? What is the probability that one hires exactly n times?
7. Given the linear program:

$$\text{maximize } (x_1 - 2x_3)$$

$$x_1 - x_2 \leq 1$$

$$2x_2 - x_3 \leq 1$$

$$x_1, x_2, x_3 \geq 0$$

Prove that the solution $(x_1, x_2, x_3) = \left(\frac{3}{2}, \frac{1}{2}, 0\right)$ is optimal.

8. *Proving NP-completeness by generalization* – For each of the problems below, prove that it is **NP**-complete by showing that it is a generalization of some **NP**-complete problem, covered in the lecture.
- (a) **MAX SAT**: Given a CNF formula and an integer g , find a truth assignment that satisfies at least g clauses.
 - (b) **DENSE SUBGRAPH**: Given a graph and two integers a and b , find a set of a vertices of G such that there are at least b edges between them.

9. A binary counter of unspecified length supports two operations: **increment** (which increases its value by one) and **reset** (which sets its value back to zero). Show that, starting from an initially zero counter, any sequence of n **increment** and **reset** operations takes time $O(n)$; that is, the amortized time per operation is $O(1)$.
10. Given two strings $x = x_1x_2 \dots x_n$ and $y = y_1y_2 \dots y_m$, the goal is to find the length of their *longest common subsequence*, that is, the largest k for which there are indices $i_1 < i_2 < \dots < i_k$ and $j_1 < j_2 < \dots < j_k$ with:

$$x_{i_1}x_{i_2} \dots x_{i_k} = y_{j_1}y_{j_2} \dots y_{j_k}.$$

Show the algorithm to achieve this goal runs in time $O(mn)$.

11. Following is a property that can be exploited for a new algorithm for finding minimum spanning trees:

Pick any cycle in the graph, and let e be the heaviest edge in that cycle. Then there is a minimum spanning tree that does not contain e .

- (a) Prove this property.
- (b) In the below-mentioned new algorithm, the input is an undirected graph $G = (V, E)$ in adjacency-list format with edge weights $\{w_e\}$.

```
sort the edges according to their weights
for each edge  $e \in E$  in decreasing order of  $w_e$ :
    if  $e$  is part of a cycle in  $G$ :
         $G = G - e$  (i.e., remove  $e$  from  $G$ )
return  $G$ 
```

Prove that this algorithm is correct.
